Spec vs impl

Equivalence checking always compares two circuits: a golden spec and a candidate implementation

The idea

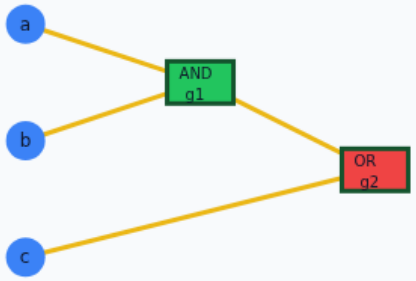
- Spec computes $f = (a \wedge b) \vee c$ with two gates

Spec netlist

SPEC VS IMPL

Spec netlist

Step 1 / 5 · spec · module01-01-spec-impl



Explanation

Spec computes f = (a∧b)∨c with two gates.

- AND then OR

```
gates=2
```

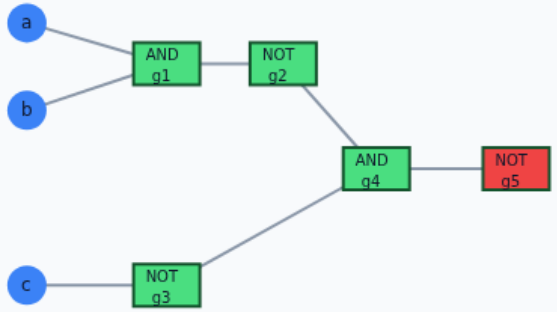
Spec netlist

Good rewrite

SPEC VS IMPL

Good rewrite

Step 2 / 5 · good · module01-01-spec-impl



Explanation

Impl good is a De Morgan form of the same function.

- NOT/AND/NOT

```
gates=5
```

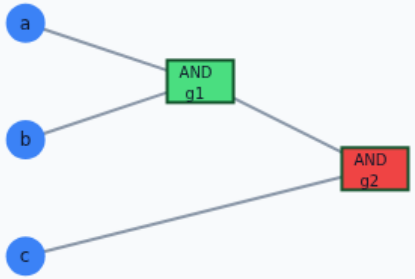
Good rewrite

Bad rewrite

SPEC VS IMPL

Bad rewrite

Step 3 / 5 · bad · module01-01-spec-impl



Explanation

Impl bad uses (a∧b)∧c — wrong when c=0.

- AND AND

```
gates=2
```

Bad rewrite

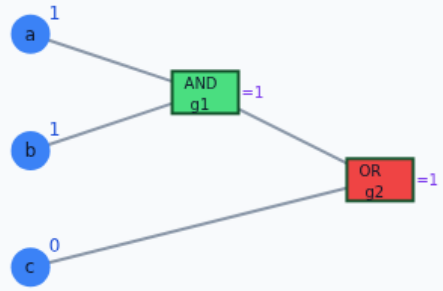
Eval at cex

SPEC VS IMPL

Eval at cex

Step 4 / 5 · eval · module01-01-spec-impl

spec=1 bad=0



Explanation

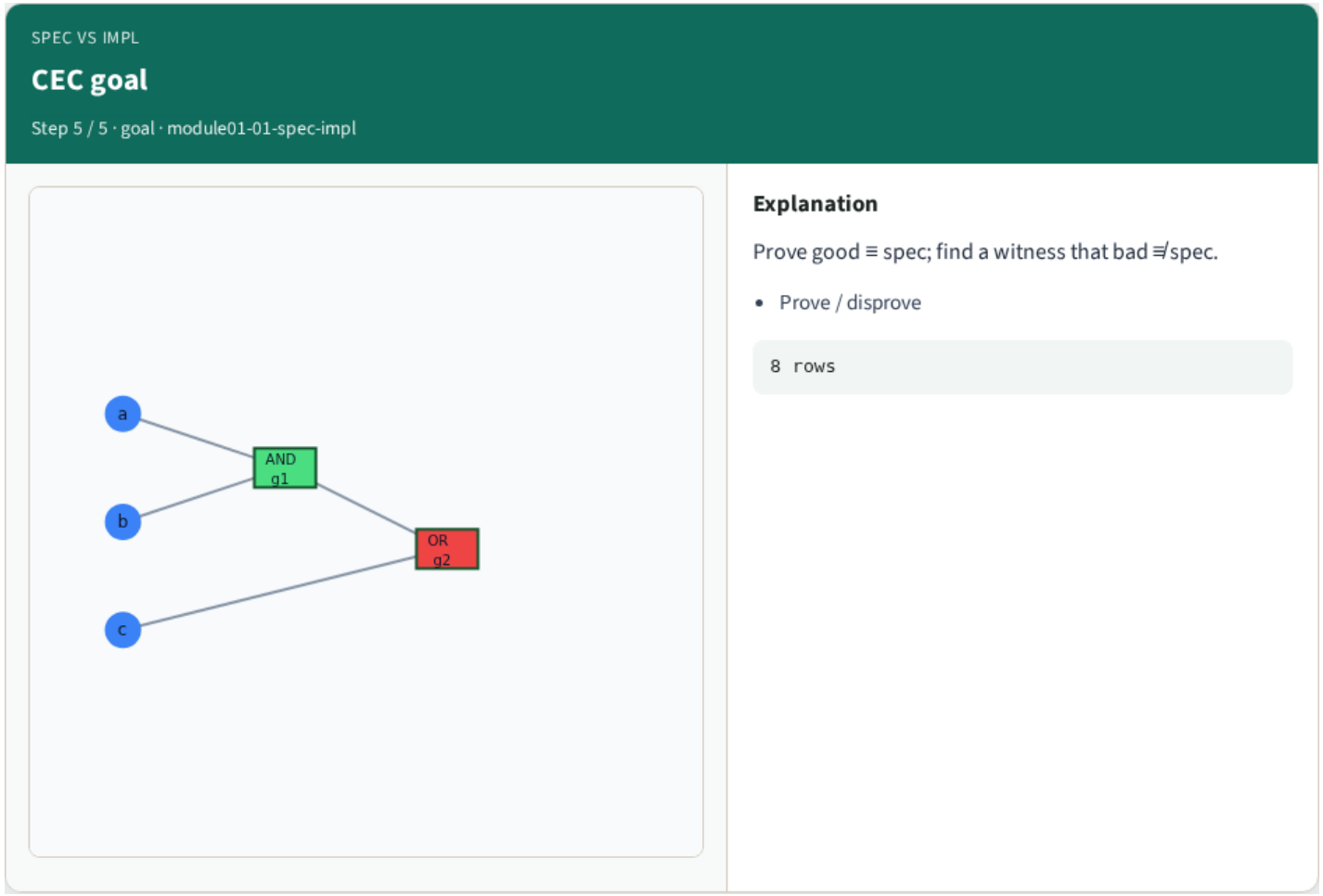
At a=1,b=1,c=0: spec=1, good=1, bad=0.

- Simulate

```
cex
```

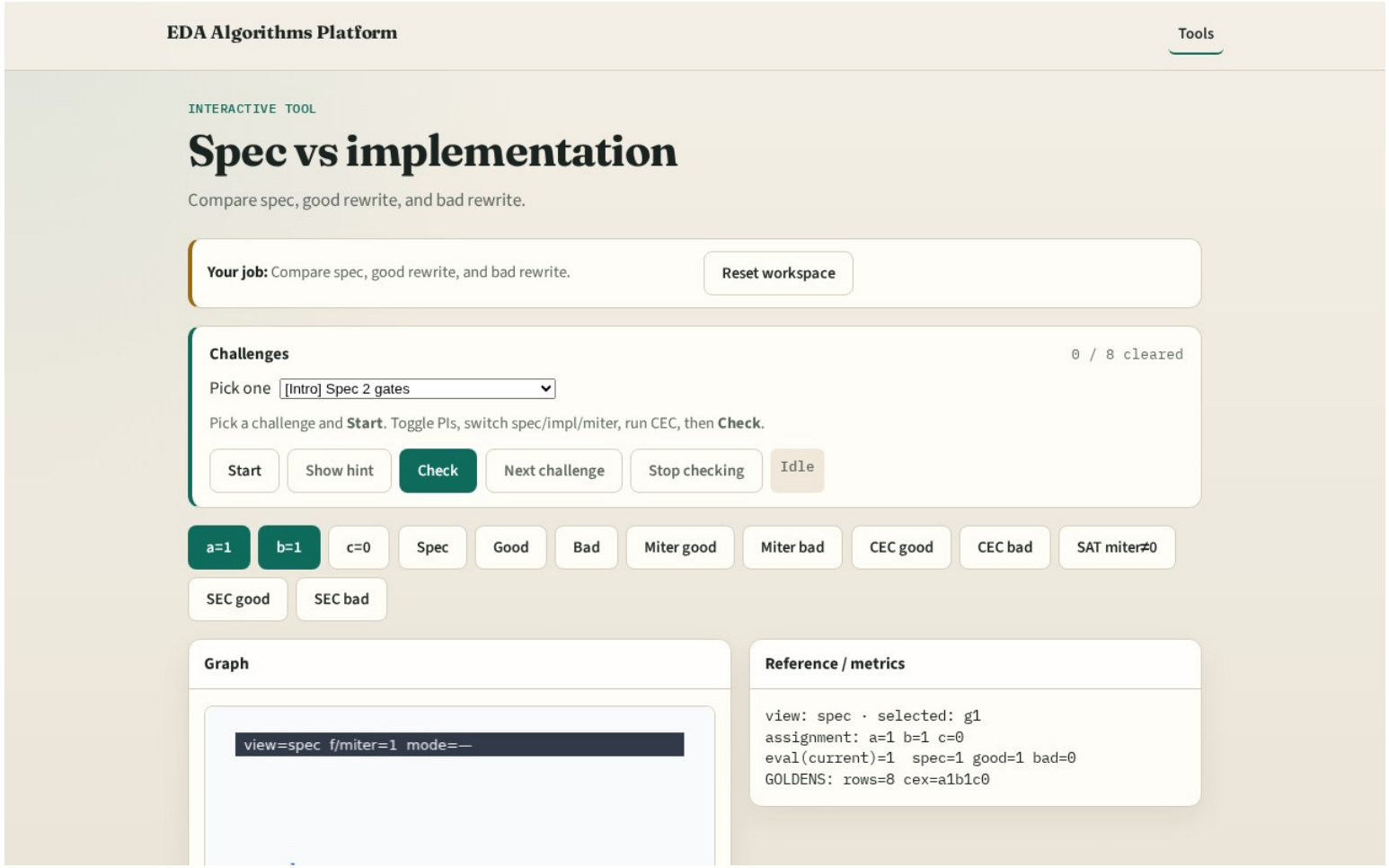
Eval at cex

CEC goal



CEC goal

Browser lab track



Browser lab starter

Implement track

- Load tiny_equiv.json; call evaluate_circuit on spec and both impls for all 8 rows

Pitfalls

- Swapping spec and impl
- Comparing only one input corner

Your turn

- Next: build the miter (XOR of outputs)